Parcial Algoritmos y Programación II - 7541 Curso Calvo (2º instancia) 17 de noviembre de 2014

Padrón: **Apellido y Nombre:**

Punteros: APROBADO - DESAPROBADO

1) Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
int main(){

  int *A, *C, *F;
  int **B;
  char *D, *E;
  int H = 66;
  char G = 'C';

  F = new int[3];

  for (int i = 0; i < 3; i++) {
      F[i] = H + i;
  }

  H++;
  A = F;
  C = new int;
  (*C) = A[1];
  A = F + 2;

  cout << (*F) << (*C) << A[0] << endl;

  B = &C;
  A = C;
  C = F + 1;
  (**B) = H + 2;

  cout <<  (*C) <<  (**B) << (*A) << endl;

  D = (char*) A;
  E = (char*) *B;
  F[1] = 70;

  cout << (*D) << (*E) << (*A) << endl;

  E = &G;
  H++;
  G = (char) H;
  H++;
  (*A)    = H;

  cout << (*E) << (*D) << G << endl;

  // liberar la memoria
  // ...

  return 0;
}
```

2. Implementar para la clase **ListaDoblementeEnlazada** el método

```
 /* pre : la lista tiene al menos tantos elementos como posicionElemento.
  * post: remueve el elemento localizado en la posición posicionElemento de la lista y lo devuelve.
  */
 T remover(unsigned int posicionElemento);
```

3. Implementar el método `intercambiar` de la clase `Trocador` a partir de las siguientes especificaciones:

```
class Trocador {
  public:
    /* post: realiza la máxima cantidad de intercambio entre los Trueques dados, respetando que el precio de un Trueque
     *       esté dentro de la toleracia indicada. Solo se pueden intercambiar cuando ambos Trueques tienen sendos Articulos que
     *       son intercambiables de acuerdo a sus categorías y categoriasIntercambiables.
     */
    void intercambiar(Lista<Trueque *> * ofertas);
}
```

```
class Trueque {
  public:
    /* post: Trueque inicializado para el oferente y articulo dados, con
     *       el porcentaje de Tolerancia indicado y sin estar asociado
     *       a ningún otro Trueque.
     */
    Trueque(string oferente, ArticuloDeIntercambio* articulo,
            float porcentajeDeTolerancia);
    /* post: devuelve el nombre que identifica al oferente
     */
    string obtenerOferente();
    /* post: devuelve el porcentaje de tolerancia sobre el precio.
     /
    float obtenerPorcentajeDeTolerancia();
    /* post: asigna a otroTrueque para intercambiar.
     */
    void intercambiarPor(Trueque* otroTrueque);
    /* post: devuelve el Trueque con el cual se intercambia o NULL si
     *       aún no se ha intercambiado con otroTrueque.
     */
    Trueque* obtenerIntercambio();

    /* post: devuelve el Artículo a intercambiar en el Trueque.
     */
    ArticuloDeIntercambio* obtenerArticulo();

    ~Trueque();
};
```

```
class ArticuloDeIntercambio {

  public:
    /* post: Artículo inicializado
     *       con la descripción, categoría y precio indicados.
     */
    ArticuloDeIntercambio(string descripción,
                          string categoria, float precio);

    /* post: devuelve la descripción del Artículo.
     */
    string obtenerDescripcion();

    /* post: devuelve la categoría a la que
     *       pertenece el Artículo.
     */
    string obtenerCategoria();

    /* post: devuelve el precio del artículo.
     */
    float obtenerPrecio();

    /* post: devuelve las categorías por la que se acepta
     *       hacer el intercambio de este artículo.
     */
    Lista<string>* obtenerCategoriasIntercambiables();
    ~ ArticuloDeIntercambio();
}
```

4. Diseñar la especificación e implementar un TDA que modele una **Deuda**. Debe proveer operaciones para:

- crear la Deuda recibiendo como parámetro su importe.
- obtenerSaldo: devuelve el importe aún no pagado de la Deuda.
- obtenerImporte: devuelve el importe inicial de la Deuda.
- pagar: recibe como parámetro un monto de dinero y lo aplica para saldar la Deuda.
- contarPagos: devuelve la cantida de pagos que se hicieron sobre la Deuda.
- calcularPagoPromedio: devuelve el promedio de los pagos realizados.

Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas